

A05-LLM-API-Standards

1 LLM API Standards

Document Version: 1.0

Generated: December 4, 2025

Standard: Large Language Model API Specifications

Status: Industry Standards (OpenAI, Anthropic)

1.1 Overview

LLM APIs provide standardized interfaces for accessing language models. Major standards include:

- **OpenAI API v1:** Industry standard REST API
- **Anthropic Claude API:** Alternative with safety focus
- **Unified Formats:** Common patterns across providers
- **Streaming:** Real-time token generation
- **Function Calling:** Tool use integration
- **Rate Limiting:** Usage management

1.1.1 Key Concepts

1. **Models:** Different capabilities and costs
 2. **Messages:** Conversation format
 3. **Tokens:** Billing unit
 4. **Temperature:** Output randomness
 5. **Top-p:** Diversity control
 6. **Stop Sequences:** Output termination
-

1.2 Authoritative References

- **OpenAI API:** <https://platform.openai.com/docs/api-reference>
 - **Anthropic Claude:** <https://docs.anthropic.com/en/docs/about-claude/models>
 - **Model Comparison:** <https://www.llmspreadsheet.com>
 - **Pricing:** <https://platform.openai.com/pricing>
-

1.3 Format Structure

1.3.1 OpenAI Chat Completions API

Endpoint:

```
POST https://api.openai.com/v1/chat/completions
Authorization: Bearer sk-...
```

Request Format:

```
{
  "model": "gpt-4-turbo",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful assistant."
    },
    {
      "role": "user",
      "content": "What is machine learning?"
    }
  ],
  "temperature": 0.7,
  "max_tokens": 1024,
  "top_p": 1.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stream": false
}
```

Response Format:

```
{
  "id": "chatcmpl-8N8Y7HI...",
  "object": "chat.completion",
  "created": 1699564200,
  "model": "gpt-4-turbo",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Machine learning is a subset of artificial intelligence..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 20,
    "completion_tokens": 150,
    "total_tokens": 170
  }
}
```

1.3.2 Anthropic Claude API

Endpoint:

```
POST https://api.anthropic.com/v1/messages
x-api-key: sk-ant-...
```

Request Format:

```
{
  "model": "claude-3-sonnet-20240229",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {
      "role": "user",
      "content": "What is machine learning?"
    }
  ],
  "temperature": 0.7,
  "top_p": 1.0
}
```

Response Format:

```
{
  "id": "msg_013Zva2CMHLNAmqNn56Rw7oF",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Machine learning is a subset of artificial intelligence..."
    }
  ],
  "model": "claude-3-sonnet-20240229",
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 20,
    "output_tokens": 150
  }
}
```

1.3.3 Function Calling Format

Tool Definition (OpenAI):

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get weather for a location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "City name"
        },
        "unit": {
          "type": "string",
          "enum": ["celsius", "fahrenheit"],
          "description": "Temperature unit"
        }
      },
      "required": ["location"]
    }
  }
}
```

Response with Tool Call:

```
{
  "role": "assistant",
  "content": null,
  "tool_calls": [
    {
      "id": "call_abc123",
      "type": "function",
      "function": {
        "name": "get_weather",
        "arguments": "{\"location\": \"San Francisco\", \"unit\": \"celsius\"}"
      }
    }
  ]
}
```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Basic Chat Completion

Python OpenAI Client:

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def chat_completion(prompt: str, model: str = "gpt-4-turbo") -> str:
    """Send chat prompt and get completion"""

    response = client.chat.completions.create(
        model=model,
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ],
        temperature=0.7,
        max_tokens=1024
    )

    return response.choices[0].message.content

# Usage
answer = chat_completion("Explain quantum computing in simple terms")
print(answer)
```

Python Anthropic Client:

```

import os
import anthropic

client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def claude_completion(prompt: str, model: str = "claude-3-sonnet-20240229") -> str:
    """Send prompt to Claude"""

    message = client.messages.create(
        model=model,
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )

    return message.content[0].text

# Usage
answer = claude_completion("Explain quantum computing in simple terms")
print(answer)

```

1.4.2 Use-Case 2: Streaming Responses

OpenAI Streaming:

```

def stream_completion(prompt: str) -> None:
    """Stream response token by token"""

    stream = client.chat.completions.create(
        model="gpt-4-turbo",
        messages=[
            {"role": "user", "content": prompt}
        ],
        stream=True
    )

    for chunk in stream:
        if chunk.choices[0].delta.content is not None:
            print(chunk.choices[0].delta.content, end="", flush=True)

    print() # NewLine

# Usage
stream_completion("Write a short story about AI")

```

Anthropic Streaming:

```
def stream_claude(prompt: str) -> None:
    """Stream Claude response"""

    with client.messages.stream(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        messages=[
            {"role": "user", "content": prompt}
        ]
    ) as stream:
        for text in stream.text_stream:
            print(text, end="", flush=True)

    print()

# Usage
stream_claude("Explain machine learning")
```

1.4.3 Use-Case 3: Function Calling (Tool Use)

OpenAI Function Calling:

```
import json

def calculator_tool(expression: str) -> float:
    """Simple calculator tool"""
    try:
        return eval(expression)
    except:
        return None

def chat_with_tools(user_message: str) -> str:
    """Chat with function calling enabled"""

    tools = [
        {
            "type": "function",
            "function": {
                "name": "calculator",
                "description": "Evaluate mathematical expressions",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "expression": {
                            "type": "string",
                            "description": "Mathematical expression to evaluate"
                        }
                    },
                    "required": ["expression"]
                }
            }
        }
    ]

    messages = [
        {"role": "user", "content": user_message}
    ]

    # Initial request
    response = client.chat.completions.create(
        model="gpt-4-turbo",
        messages=messages,
        tools=tools,
        tool_choice="auto"
    )

    # Handle tool calls
```

```

while response.choices[0].finish_reason == "tool_calls":
    tool_call = response.choices[0].message.tool_calls[0]

    if tool_call.function.name == "calculator":
        args = json.loads(tool_call.function.arguments)
        result = calculator_tool(args["expression"])

        # Add assistant response and tool result
        messages.append(response.choices[0].message)
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": str(result)
        })

        # Get next response
        response = client.chat.completions.create(
            model="gpt-4-turbo",
            messages=messages,
            tools=tools
        )

    return response.choices[0].message.content

# Usage
answer = chat_with_tools("What is 2^10 + 5*3?")
print(answer)

```

Claude Tool Use:

```

def claude_with_tools(user_message: str) -> str:
    """Claude with function calling"""

    tools = [
        {
            "name": "weather_api",
            "description": "Get weather for a location",
            "input_schema": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "City name"
                    }
                },
                "required": ["location"]
            }
        }
    ]

    messages = [
        {"role": "user", "content": user_message}
    ]

    # Initial request
    response = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        tools=tools,
        messages=messages
    )

    # Process tool calls
    while response.stop_reason == "tool_use":
        tool_use = next(
            (block for block in response.content if block.type == "tool_use"),
            None
        )

        if tool_use:

```

```

if tool_use:
    # Execute tool
    result = {"location": tool_use.input["location"], "temp": "72F"}

    # Continue conversation
    messages.append({"role": "assistant", "content": response.content})
    messages.append({
        "role": "user",
        "content": [
            {
                "type": "tool_result",
                "tool_use_id": tool_use.id,
                "content": json.dumps(result)
            }
        ]
    })

    response = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        tools=tools,
        messages=messages
    )

# Get final text
text_content = next(
    (block for block in response.content if hasattr(block, "text")),
    None
)

return text_content.text if text_content else "No response"

# Usage
answer = claude_with_tools("What's the weather in NYC?")
print(answer)

```

1.5 Examples

1.5.1 Example 1: Model Pricing and Selection

Pricing Comparison (as of 2024):


```

{
  "models": [
    {
      "name": "gpt-4-turbo",
      "input_cost_per_1m_tokens": 10,
      "output_cost_per_1m_tokens": 30,
      "context_window": 128000,
      "capabilities": ["vision", "function_calling", "json_mode"]
    },
    {
      "name": "gpt-4o",
      "input_cost_per_1m_tokens": 5,
      "output_cost_per_1m_tokens": 15,
      "context_window": 128000,
      "capabilities": ["vision", "function_calling", "json_mode"]
    },
    {
      "name": "gpt-3.5-turbo",
      "input_cost_per_1m_tokens": 0.50,
      "output_cost_per_1m_tokens": 1.50,
      "context_window": 4096,
      "capabilities": ["function_calling"]
    },
    {
      "name": "claude-3-opus",
      "input_cost_per_1m_tokens": 15,
      "output_cost_per_1m_tokens": 75,
      "context_window": 200000,
      "capabilities": ["vision", "function_calling"]
    },
    {
      "name": "claude-3-sonnet",
      "input_cost_per_1m_tokens": 3,
      "output_cost_per_1m_tokens": 15,
      "context_window": 200000,
      "capabilities": ["vision", "function_calling"]
    }
  ]
}

```

1.5.2 Example 2: Error Handling

```

def robust_api_call(prompt: str, max_retries: int = 3) -> str:
    """Call API with error handling and retry logic"""

    import time
    from openai import RateLimitError, APIError

    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model="gpt-4-turbo",
                messages=[{"role": "user", "content": prompt}],
                timeout=30
            )
            return response.choices[0].message.content

        except RateLimitError as e:
            if attempt < max_retries - 1:
                wait_time = 2 ** attempt # Exponential backoff
                print(f"Rate limited. Retrying in {wait_time}s...")
                time.sleep(wait_time)
            else:
                raise

        except APIError as e:
            print(f"API error: {e}")
            if attempt < max_retries - 1:
                time.sleep(1)
            else:
                raise

```

1.5.3 Example 3: Cost Estimation

```
import tiktoken

def estimate_cost(messages: list, model: str = "gpt-4-turbo") -> dict:
    """Estimate API cost"""

    # Get tokenizer
    encoding = tiktoken.encoding_for_model(model)

    # Count input tokens
    input_tokens = 0
    for message in messages:
        tokens = encoding.encode(message["content"])
        input_tokens += len(tokens)

    # Pricing (update as needed)
    pricing = {
        "gpt-4-turbo": {"input": 0.01, "output": 0.03},
        "gpt-4o": {"input": 0.005, "output": 0.015},
        "gpt-3.5-turbo": {"input": 0.0005, "output": 0.0015}
    }

    rates = pricing.get(model, pricing["gpt-4-turbo"])

    # Estimate
    input_cost = (input_tokens / 1000) * rates["input"]

    # Assume output ~= 1.3x input for typical responses
    estimated_output = int(input_tokens * 1.3)
    output_cost = (estimated_output / 1000) * rates["output"]

    return {
        "input_tokens": input_tokens,
        "estimated_output_tokens": estimated_output,
        "estimated_input_cost": f"${input_cost:.4f}",
        "estimated_output_cost": f"${output_cost:.4f}",
        "estimated_total_cost": f"${input_cost + output_cost:.4f}"
    }

# Usage
messages = [{"role": "user", "content": "Explain quantum computing"}]
cost = estimate_cost(messages)
print(cost)
```

1.6 Model Comparison

Model	Speed	Cost	Quality	Context	Best For
GPT-4 Turbo	Medium	High	Excellent	128K	Complex reasoning
GPT-4o	Fast	Medium	Excellent	128K	Balanced, fast
GPT-3.5	Very Fast	Low	Good	4K	Simple tasks
Claude-3-Opus	Slow	High	Excellent	200K	Long documents
Claude-3-Sonnet	Medium	Medium	Very Good	200K	Balanced

1.7 Tools & Ecosystem

Tool	Purpose	URL
OpenAI SDK	Python client	https://github.com/openai/openai-python
Anthropic SDK	Claude client	https://github.com/anthropics/anthropic-sdk-python
LiteLLM	Multi-provider	https://litellm.vercel.app
Langchain	Orchestration	https://docs.langchain.com

	Tool	Purpose	URL
Prompt Flow		Debugging	https://github.com/microsoft/promptflow

Navigation: [Back to Index](#) | [Previous: Prompt Engineering Standards](#)